

速查表(单页 CheatSheet)

用途:做题时反复翻这一页 > 翻文档。

范围:输入输出 3 套模板 + 13 容器 API 对照 + 8 常用代码片段 + 关键字速查 + 15 大坑点提醒。

前提:假设你已读完 00-07,本表只浓缩,不解释。

一、输入输出 3 套模板

1.1 ACM 模式完整骨架(牛客)

```
import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StreamTokenizer in = new StreamTokenizer(br);
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
        // 业务
        out.flush();
    }
}
```

1.2 多组 T 输入 / 一行 N 个数

```
in.nextToken(); int T = (int) in.nval;    // 读 T
while (T-- > 0) {
    in.nextToken(); int n = (int) in.nval;    // 读 N
    in.nextToken(); int sum = 0;              // ★ 复用 一个变量
    for (int i = 0; i < n; i++) {
        in.nextToken(); sum += (int) in.nval;    // 读 N 个数
    }
    out.println(sum);
}
```

详见:01 输入输出。

1.3 LeetCode 模式骨架

```
class Solution {
    public 返回类型 方法名(参数类型 参数) {
        // 业务
        return 结果;
    }
}
```

二、13 容器 API 一行对照(精简版)

C++ STL	Java	关键差异	常用 API 映射
<code>`vector<T>`</code>	<code>`ArrayList<T>`</code>	基本类型用包装	<code>`push_back→add`</code> / <code>`size→size`</code> / <code>`[]→get/set`</code>
<code>`list<T>`</code>	<code>`LinkedList<T>`</code>	慢,优先 ArrayDeque	<code>`push_back→add`</code>
<code>`stack<T>`</code>	<code>`ArrayDeque<T>`</code>	别用 <code>`Stack`</code>	<code>`push→push`</code> / <code>`top→peek`</code> / <code>`pop→pop`</code>
<code>`queue<T>`</code>	<code>`ArrayDeque<T>`</code>	推荐 ArrayDeque	<code>`push→offer`</code> / <code>`front→peek`</code> / <code>`pop→poll`</code>
<code>`priority_queue<T>`</code>	<code>`PriorityQueue<T>`</code>	默认小顶堆!	大顶堆: <code>`(a,b)→b-a`</code> 或 <code>`Collections.reverseOrder()`</code>
<code>`deque<T>`</code>	<code>`ArrayDeque<T>`</code>	滑动窗口标配	<code>`addFirst/Last`</code> / <code>`peekFirst/Last`</code> / <code>`pollFirst/Last`</code>
<code>`unordered_map<K,V>`</code>	<code>`HashMap<K,V>`</code>	允许 null	<code>`[]→get/put`</code> / <code>`find→containsKey`</code>
<code>`unordered_set<T>`</code>	<code>`HashSet<T>`</code>	基于 HashMap	<code>`insert→add`</code> / <code>`find→contains`</code>
无直接对应	<code>`LinkedHashMap<K,V>`</code>	LRU 缓存	<code>`new LinkedHashMap<>(cap, 0.75f, true)`</code>
<code>`map<K,V>`</code>	<code>`TreeMap<K,V>`</code>	红黑树	<code>`lower_bound→lowerKey`</code> / <code>`upper_bound→higherKey`</code>
<code>`set<T>`</code>	<code>`TreeSet<T>`</code>	红黑树	<code>`lower_bound→lower`</code>
<code>`pair<T1,T2>`</code>	<code>`int[]`</code> / <code>`Map.Entry`</code> / 自定义类	无内建 pair	
<code>`sort`</code>	<code>`Arrays.sort`</code> / <code>`List.sort`</code>		稳定: <code>`Arrays.sort(对象)`</code>

详见:03 容器与 STL 对照。

三、8 常用代码片段(直接抄)

3.1 排序 + 自定义比较器

```
Arrays.sort(arr); // 升序(基本类型)
Arrays.sort(arr, (a, b) -> b - a); // 降序
Arrays.sort(points, Comparator.comparingInt((int[] p) -> p[1]).thenComparingInt(p -> p[0])); // 多键
```

3.2 二分(找左边界)



```
int lo = 0, hi = arr.length;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid;
}
return lo; // 第一个  $\geq$  target 的位置
```

3.3 双指针(对撞)

```
int lo = 0, hi = arr.length - 1;
while (lo < hi) {
    int sum = arr[lo] + arr[hi];
    if (sum == target) return new int[]{lo, hi};
    else if (sum < target) lo++;
    else hi--;
}
```

3.4 BFS(层序)

```
ArrayDeque<TreeNode> q = new ArrayDeque<>();
q.offer(root);
while (!q.isEmpty()) {
    int sz = q.size(); // ★ 提前取
    for (int i = 0; i < sz; i++) {
        TreeNode node = q.poll();
        // 业务
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
    }
}
```

3.5 DP(一维)

```
int[] dp = new int[n];
dp[0] = 1; dp[1] = 1;
for (int i = 2; i < n; i++) dp[i] = dp[i-1] + dp[i-2];
return dp[n-1];
```

3.6 并查集

```
int[] parent = new int[n];
for (int i = 0; i < n; i++) parent[i] = i;
int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
void union(int x, int y) { parent[find(x)] = find(y); }
```

3.7 单调栈(下一个更大,LeetCode 496/739 风格)



```

ArrayDeque<Integer> st = new ArrayDeque<>(); // 存下标
int[] res = new int[arr.length];
Arrays.fill(res, -1);
for (int i = 0; i < arr.length; i++) {
    while (!st.isEmpty() && arr[st.peek()] < arr[i]) {
        res[st.pop()] = arr[i];
    }
    st.push(i);
}

```

3.8 回溯(三件套)

```

List<List<Integer>> result = new ArrayList<>();
void backtrack(int[] nums, List<Integer> path, boolean[] used) {
    if (path.size() == nums.length) { result.add(new ArrayList<>(path)); return; }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;
        path.add(nums[i]); used[i] = true;
        backtrack(nums, path, used);
        path.remove(path.size() - 1); used[i] = false;
    }
}

```

四、关键字速查

关键字	含义	刷题常见
`final`	常量(修饰变量/引用/方法)	`final int N = 100;` (常量)
`static`	类共享(非实例)	`static ListNode head;` (全局节点)
`this`	当前实例引用	`this.val = val;` (构造器)
`super`	父类引用	`super(val);` (调用父类构造)
`@Override`	标注重写方法	重写 `compareTo` / `compare`

五、15 大坑点一句话提醒(救命)

#	一句话	#	一句话
1	`Stack` 用 `ArrayDeque` 替	9	`int[]` 在 `Arrays.asList` 行为不同
2	`PriorityQueue` 默认小顶堆(反 C++)	10	`HashMap` 多线程用 `ConcurrentHashMap`
3	`Integer` `==` 在 128 外失效,用 `equals`	11	`String` `==` 比引用,用 `equals`
4	遍历集合 `remove` 抛异常,要用迭代器	12	`arr.length` / `s.length()` / `list.size()` 三种语法



5	`Arrays.asList` 返回定长 list	13	数组默认 0/false/null
6	字符串拼接用 `StringBuilder`	14	深递归爆栈(默认栈小)
7	浮点比较用 `Math.abs(a-b) < eps`	15	`main` 必须 `throws IOException`
8	二维数组排序要自定义 `Comparator`		

详见:06 常见坑点速查。

六、8 篇文档导航

#	文档	一句话
00	总览与速成路线	路线图
01	输入输出篇	ACM 模式骨架
02	语法差异速览	纯语法
03	容器与 STL 对照	核心
04	常用工具类	Java 8 必备
05	刷题模式与模板	10 大算法
06	常见坑点速查	救命文档
07	实战例题	7 道典型题

